

Estrutura de Dados — Lista de Exercícios (Aula 02)

Análise de Complexidade, Vetores (Array) e Funções/Métodos em C# — Exercícios (Bloom 1 e 2)

Obs: Não precisa entregar, mas vai ajudar muito nas provas NI e PO.

Parte A — Exercícios Bloom 1 (Lembrar) — 5 questões

1. Defina “análise empírica” e “análise analítica” do tempo de execução de um algoritmo.
2. No algoritmo de achar o maior entre 3 números, quantos passos são atribuídos no exemplo da aula?
3. O que significa dizer que um algoritmo tem complexidade $O(1)$? Dê um exemplo compatível com a aula.
4. Liste 4 características de um vetor (array) em C# apresentadas/compatíveis com a aula.

Parte B — Exercícios Bloom 2 (Compreender/Aplicar) — 3 questões

5. O que representa um algoritmo com tempo de execução $O(2^N)$?
6. Para o algoritmo que soma os N primeiros inteiros, explique por que a constante “+3” se torna irrelevante quando n é grande e conclua a ordem em Big-O.
7. Considere um array lista de tamanho N e um laço que percorre do índice 0 até lista.Length-1 imprimindo os elementos. Justifique por que o tempo é $O(N)$.
8. No trecho que cria e preenche um vetor com números aleatórios e depois imprime (dois for), identifique quais linhas “dominam” a complexidade e determine a ordem em Big-O.

Parte C — Implementação (Vetor em C# com Funções) — 2 exercícios**Exercício 09 — Preencher, imprimir e somar pares****Descrição:**

Crie um vetor `int[] v` de tamanho N (fixo no código, por exemplo $N = 10$).

Preencha o vetor com números aleatórios entre 1 e 100.

Imprima todos os elementos do vetor em uma única linha.

Calcule e imprima a soma dos valores pares contidos no vetor.

Requisitos (funções obrigatórias):

```
static void PreencherVetor(int[] v, int min, int max)
```

```
static void ImprimirVetor(int[] v)
```

```
static int SomarPares(int[] v)
```

Saída esperada (exemplo):

Vetor: 12 7 90 3 44 21 8 10 1 33

Soma dos pares: 164

Exercício 10 — Buscar valor e contar ocorrências**Descrição:**

Crie um vetor `int[] v` de tamanho `N` (fixo no código, por exemplo `N = 15`).

Preencha o vetor com números aleatórios entre 0 e 9 (para haver repetições).

Imprima o vetor em uma única linha.

Leia um número `x` (entre 0 e 9) digitado pelo usuário.

Informe a primeira posição (índice) em que `x` aparece (ou -1 se não aparecer).

Informe quantas vezes `x` aparece no vetor.

Requisitos (funções obrigatórias):

```
static void PreencherVetor(int[] v, int min, int max)
```

```
static void ImprimirVetor(int[] v)
```

```
static int PrimeiraOcorrencia(int[] v, int x)
```

```
static int ContarOcorrencias(int[] v, int x)
```

Saída esperada (exemplo):

Vetor: 3 9 0 3 1 3 7 8 3 2 3 5 6 3 4

Digite `x` (0 a 9): 3

Primeira ocorrência: 0

Quantidade: 6

Observação: utilize `for` para percorrer o vetor e `Random` para gerar valores. As funções devem receber o vetor por parâmetro, conforme os exemplos vistos em aula.